On Software Reference Architectures and Their Application to the Space Domain

Marco Panunzio* and Tullio Vardanega

University of Padova
Department of Mathematics
via Trieste 63, 35121 Padova, Italy
{panunzio,tullio.vardanega}@math.unipd.it

Abstract. In high-integrity systems a rising portion of software assets and development activities address quality and conformance issues in several non-functional dimensions. For those systems the software architecture acquires a prominent role: it does in fact express the framework that hosts the required functionalities, while the principles and guarantees that underpin its definition assure the desired non-functional quality on the software product. A software reference architecture holds for a set of systems and prescribes the form that concrete software architectures have to have for those systems. The software reference architecture can thus be seen as a generic software architecture, whose assets are recognized by domain stakeholders as befitting the construction of a given class of systems, for which they have been proven to meet the applicable industrial needs and technical requirements. This paper discusses the rationale for the understanding and definition of a software reference architecture and present its use in an initiative promoted by the European Space Agency for its future satellite systems.

1 Introduction

High-integrity systems must fulfil stringent requirements in a number of *non-functional* dimensions, such as timing predictability, dependability, and, more recently, security [16]. For those systems, the *software architecture* becomes the cornerstone for realizing a software product that provides all the required functionality while fulfilling all the non-functional requirements.

A software reference architecture may be regarded as a software architecture that applies to the realization of a certain class of software systems. It is fixed after a thorough domain analysis and becomes a solution recognized by the domain stakeholders for the fulfillment of the industrial needs of interest in that domain.

The work presented in this paper was prompted by an initiative undertaken by the European Space Agency (ESA), to promote the establishment of a software reference architecture for use among its industrial suppliers in the development of the on-board software of their future satellite systems. This paper discusses why the concepts of software architecture and the software reference architecture were central to that effort and

* This author is now with Thales Alenia Space - France.

J. Favaro and M. Morisio (Eds.): ICSR 2013, LNCS 7925, pp. 144–159, 2013.

formed a solid basis on which the industrial needs of interest to the domain stakeholders could demonstrably be met. It provides a comprehensive articulation of the principles that guided that initiative, narrated by authors who were directly involved in it from its outset, with the full concurrence of all the relevant industrial actors.

The remainder of the paper is organized as follows: section 2 discusses the role of software architecture in high-integrity software development, and argues why it is crucial to the satisfaction of the governance of the industrial domain; section 3 illustrates the consequences ensuing from elevating a software architecture in an industrial domain to the status of reference architecture for that domain; section 4 outlines the context of the European space market, discusses the industrial needs placed on future missions of ESA, and relates them to the definition of a software reference architecture for that context; section 5 presents the cornerstones on which the ESA software reference architecture was built; and finally, section 6 draws some conclusions.

2 The Role of the Software Architecture

One disconcerting situation in the software engineering practice is the exceedingly informal and liberal interpretation of the concept of *software architecture*. A brilliant exercise carried out by the Software Engineering Institute [22] testifies this confusion, which is patently at odds with the centrality of that concept for the discipline.

The main misconception is that software architecture is often liberally used as a synonym of *software design*. Software design instead is just one of the concerns addressed by the software architecture, whose overarching role is to govern the software and the process by which it will be built and operated. More importantly, the software architecture deals with the *principles* guiding the design and evolution of a software system. This aspect is central and *precedes in importance* software design.

The definition of architecture given in the IEEE 1471 standard, later adopted and approved by ISO/IEC as ISO 42010 [12], was singled out as the reference for this work, as the best and most authoritative one: *"The fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution"*.

This definition helps singling out the concerns that are addressed by the software architecture [17], which figure 1 captures diagrammatically with a numbering that we will use later for tracing the proposed solution to the corresponding concerns:

- *Software decomposition*: the organization of the software in terms of parts, so that every individual part has its own architectural cohesiveness and the interactions between parts are minimized so as to reduce coupling;
- *Externally visible attributes of software "components"*: the attributes of those software parts ("components" in our case) that are externally visible. They represent features or needs specific to the component that can influence other parts of the software or its overall properties. The other internal attributes shall remain invisible to the outside with no influence on the software architecture;
- *Relationship between software "components"*: how components relate to one another to provide services and fulfil needs;

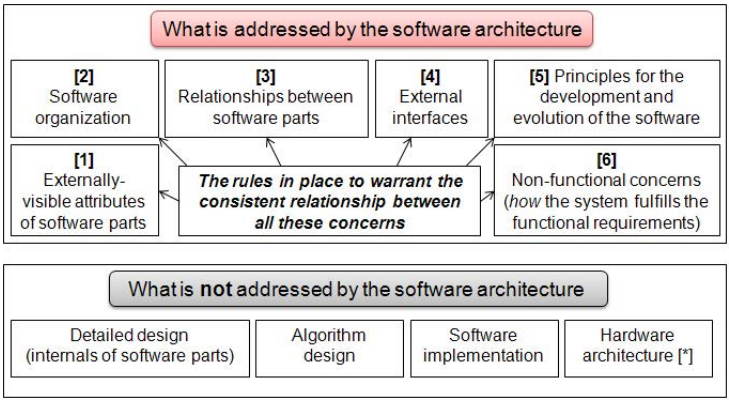

Fig. 1. The concerns addressed by the software architecture and those outside its perimeter

- *Non-functional concerns*: the abstraction level where most non-functional concerns are best addressed;
- *External interfaces*: the way the software interacts with the external environment (e.g. by commanding sensors or actuators, or serving external interrupts);
- *Principles for the development and evolution of the software*: the software design process fixes the rules for its development and provides means to perform software maintenance and evolution and dictates the supported form of reuse;
- *The rules in place to warrant the consistent relationship between all the above concerns*: every software system descends (at least implicitly) from some software architecture [12] ; however, to establish a software architecture that truly supports the given goals, a methodology must be defined that underpins the development approach and warrants consistency for all the aspects mentioned above.

Interestingly, the following aspects are not of pertinence to a software architecture:

- *Detailed design*, which is the refinement of the software decomposition performed at architectural level by providing the internal organization of components;
- *Algorithm design*, which is the design of the algorithmic behaviour of the software to fulfill the functional requirements;
- *Software implementation*, which is the activity for the implementation of software;
- *Hardware architecture*, which is addressed completely outside of the the software architecture concerns and yet needs to present a description of its essential characteristics, constraints and limitations to the latter, especially for those hardware aspects that concern communication and feasibility analysis.

3 The Concept of Reference Architecture

Much like for "software architecture", also the concept of *reference architecture* misses a single agreed definition. The gloss that makes the most hits is probably that provided

by the "Rational Unified Process" (RUP) [13]: *"a predefined architectural pattern, or set of patterns, possibly partially or completely instantiated, designed, and proven for use in particular business and technical contexts, together with supporting artefact to enable their use. Often, these artefacts are harvested from previous projects."* Though generic, the cited definition includes interesting aspects, which deserve discussion.

First, we should clarify the difference between a software architecture and a software reference architecture. The software reference architecture prescribes the form of the concrete software architectures for a set of systems for which it was developed. Arguably, the reference architecture is a form of "generic" software architecture, which prescribes the founding principles, the underlying methodology and the architectural practices that were recognized by the domain stakeholders as the best solution to the construction of a certain class of software systems. The architecture of one target software system can then be considered as an "instantiation" to the specific system needs of the software reference architecture.

The other interesting element in the citation is that a reference architecture is "proven for use in particular business and technical contexts". This observation underlines:

1. the importance of the elicitation of the industrial needs and technical requirements from which the reference architecture emanate;
2. the need for *validation in use* as well as for some quantifiable *evaluation* of the reference architecture. Validation demonstrates empirically that software architectures resulting from application of the software reference architecture to concrete systems satisfactorily fulfill all the industrial needs that originated it. Evaluation ascertains the goodness of the reference architecture by analysing to what extent it satisfies the industrial needs, and can be facilitated by appropriate methodological support, for example the Architecture Tradeoff Analysis Method [11] (though it would require some adaptation to apply to the level of reference architecture [2]).

There exist multiple kinds of reference architectures. Angelov et al. [1] propose a characterisation in terms of *context*, *goal* and *design*. The context dimension discriminates: *where* the reference architecture will be used (i.e., within a single organization or multiple organizations), *who* defines it (i.e., user organizations, software organizations, research centres, standardization bodies), *when* is it defined (a *preliminary* reference architecture is defined before an implementation of it exists, or *classical* when experience on the application on the target class of systems has already been acquired). The goal dimension describes the main usage of the reference architecture: it may be a *standardization* of existing software architectures, or a *facilitation* for the design of concrete architectures (by providing guidelines, methodologies and patterns of the software architecture). The design dimension describes the main design choices of the reference architecture: *what* is described (components, interfaces, protocols, guidelines, etc.), the *abstraction level* of the description (concrete, semi-concrete, or abstract; corresponding to decreasing levels of dependence with technological and implementation decisions) and the level of formality of the description (*informal, semi-formal, formal*).

4 An Application Case

European space industry has entered an economic climate in which the funding availed to future missions is tightly capped. To win the day, proponents are therefore compelled to take on increasingly more ambitious scientific challenges. The resulting growth in complexity is thus plainly at odds with capped budgets.

One of the major consequences of this situation is that the activities that contribute the most to the value added of the mission, i.e., mission analysis and system engineering, will play an even bigger role in the overall economy of the project, with a proportional increase of the time and cost invested on them.

This situation calls for a rise in the *cost-effectiveness* of software development, to increase the ”value” of the software product delivered with a given budget. The best way to succeed in this challenge is to increase the efficiency of software development by singling out the recurring costs and abating them as much as possible.

It is therefore necessary to first understand where this can be done, i.e., to identify which factors can be intervened on and with what degree of freedom; and subsequently, to investigate the definition of a solution.

The European Space Agency (ESA) has recently decided to cope with this emerging scenario with the definition, realization and adoption of a software reference architecture for the development of on-board software for satellites.

4.1 Domain Analysis

A typical satellite system includes two main constituent parts: the payload and the service module (also known as bus, or platform) [10]. The payload part comprises the instruments necessary for the scientific mission (telescopes, spectrometers, detectors, etc..). The service module is used to govern the satellite position, orientation and manoeuvres, communicate with ground, and ensure that the thermal and power needs are satisfied; for this reason the service module is equipped with various sensors (gyroscopes, Earth sensors, magnetometers, thermistors, etc...), actuators (reaction wheels, thrusters, heaters, etc..) and other equipment (e.g., solar arrays, batteries).

The satellite payload is mission-specific: hence, it will differ in form and instruments from one mission to another. Conversely, the service module may present a recurrent structure (i.e., shape and decomposition in subsystems) in several missions.

The on-board software of a satellite conceptually mirrors the same (physical) separation of the satellite structures. Accordingly, the on-board software can be conceptually split in two parts: the payload software and the platform software. Also in this case, the payload software is vastly – if not completely – different from one mission to the other; the platform software may instead assume a recurrent organization, which in software engineering parlance, is the *software architecture*.

The platform software comprises a set of traditional *functional contents*: the Attitude and Orbit Control System (AOCS) or a Guidance, Navigation and Control (GNC) system [15] of a deep-space mission; the Data Handling System (DHS); the thermal management; the power management; etc.. Additionally it may require the inclusion of more advanced functions that realize the functional requirements of future missions, such as advanced autonomy and planning or formation flying.

On-board software for satellites can be classified as high-integrity software; its realization is therefore subject to stringent requirements at both process and product level in dimensions such as: time and space predictability, safety, dependability, security. As a consequence, the output of the various stages of the development process are strictly regulated by domain-specific standards (such as ECSS-E-ST-40C [8] on software development and ECSS-Q-ST-80C [9] on software product quality). Moreover, the software product is subject to extensive verification and validation campaigns to to determine that it performs as expected while fulfilling all *non-functional* requirements.

The software architecture can come of use especially under cost-reduction constraints, as it (i) expresses the architectural framework that can best host the required functional contents, and (ii) enforces conformance to architectural properties and methodological principles that most contribute to the attainment of the required product quality.

The European space domain is strongly influenced by the historical structure of the industrial market, which used to be dominated by Astrium and Thales Alenia Space. For various reasons those two competitors adopted different concepts, methodologies and technologies for on-board software development, resulting in distant production styles and strategies: at the present state, a software supplier can competitively bid as a subcontractor only in a single supply chain, with adverse effects on the market economy. Whereas new and smaller prime contractors have lately made their way into the European space market, their role is less relevant to this discussion, as they operate within business conditions that currently favor opportunistic solutions over consolidation.

The ESA software reference architecture is also expected to respond to this rupture. Furthermore, the novel approach shall be able to accommodate and control the *incremental* and *iterative* development models that are inherent to the space domain. The on-board software is not simply a product to be released at the end of the development: incremental releases of it are needed to perform additive hardware/software integration and validation activities, in strict schedule coordination with other satellite development teams. Iterations are determined by the rectification, improvement and eventual finalization of system and software requirements experienced by every project, with carried risk of destructive backtracks of design and implementation decisions.

4.2 Industrial Needs

The ESA initiative for the definition of a software reference architecture is a harmonization of methodologies and technologies around the Agency charter, seeking to earn relevant benefits for all involved stakeholders: ESA as the procurement agency, software prime contractors and software suppliers. Not surprisingly, a very similar strategy was taken by NASA [6], which recommended investing on software architecture in general and software reference architectures in particular to cope with their needs for future missions. Under ESA sponsorship, one of the authors of this work devoted his entire PhD project [17] to all the essential challenges of this initiative. Much of the theoretical, methodological and technological results of that effort were first spun-in by ESA after clearance by a working group comprised of experts from ESA, software primes and suppliers, and then consolidated in the ESA-funded COrDeT-2 study project[1].

[1] `http://cordet.gmv.com`

The first input to the definition of the reference architecture consisted in gathering all the industrial needs that ESA or other stakeholders set as strategic goals.

Table 1 outlines the industrial needs. Some of those needs are common to all software domains; a few others are in common or similar to the needs of other high-integrity domains; a sizeable subset of them are instead specific to the European space domain.

Table 1. The main industrial needs of the European space domain. Type C denotes a need common to all software domains; type HI a need similar or common to other high-integrity domains; for type DS the need is domain specific.

ID	Industrial need	Type
IN-01	*Reduced software development schedule*	C
IN-02	*Higher cost-effectiveness of software development*	C
IN-03	*Support for incremental and parallel software development*	C
IN-04	*Multi-team development and product policy*	HI/DS
IN-05	*Quality of the software product*	HI
IN-06	*Lower effort intensiveness of Verification and Validation*	HI
IN-07	*Role of software suppliers*	DS
IN-08	*Mitigation of the impact of late requirement definition or change*	HI
IN-09	*Simplification and harmonization of Fault Detection Isolation and Recovery*	DS
IN-10	*Lower effort for flight maintenance*	DS
IN-11	*Future needs*	HI/DS

IN-01: Reduced Software Development Schedule. Future projects require software to be developed in a shorter schedule. The definition and finalization of software requirements occur later in the project schedule than in the past as the mission definition and system engineering phases take longer because of their greater economic incidence to the final value added. As the release of the product is usually tied to astronomical events which determine the launch window or other external factors (for example, the procurement of the launcher vehicle), the implementation activities, including software development, are compressed within the residual time toward the tail-end of development.

Moreover, staggered incremental releases of the on-board software are required so that electrical integration and HW/SW integration tests can be performed incrementally and the relevant workmanship is more efficiently deployed.

For all these reasons, although the software itself only accounts for a modest fraction of the total cost of the satellite, delayed software releases may have inordinately costly impact on the overall project schedule.

Even though the reasons that generate this industrial need are specific to the space domain, the call for reduced development schedule and shorter time-to-market is common to all software domains.

IN-02: Higher Cost-Effectiveness of Software Development. Not only will the software development budget not rise in the foreseeable future but it may instead decline in favour of other cost elements. Yet, the performance and complexity of core functions of the satellite platform may grow in response to end-user demands, while new complex functions may also be required (i.e., formation flying, advanced autonomy, etc.).

This need is common to all software domains; the means to fulfill it shall be specific to the space domain and in accord with all the other industrial needs.

IN-03: Support for Incremental and Parallel Software Development. The novel approach shall accommodate different system and software development practices and also support common, established needs.

The on-board software is built incrementally (cf. [14]), gradually adding more functionalities to an early initial release that enable electrical and avionic integration tests. On-board software is also built in *parallel development*, whereby distinct teams, possibly under different organizations, develop parts of the software system. The novel development approach shall not get in the way of the desired development model and instead help facilitate cross-team and cross-organization dialogue.

This industrial need may also apply to other high-integrity domains.

IN-04: Multi-team Development and Product Policy. Easy and clear-cut decomposition of the software product to facilitate subcontracting is crucial to the geopolitical economy of the European space domain. This need originates from the *geographical return* policy sanctioned in the ESA charter, which requires to "ensure that all Member States participate in an equitable manner, having regard to their financial contribution, in implementing the European space programme". Multiple demands descend from this need: (1) enforcement of subcontracting schemes to software primes or software suppliers only; (2) subcontracting of distinct processing units to different companies (also part of IN-03), including the responsibility for the deployed software; (3) subcontracting from software primes to software suppliers by enabling the maximum flexibility in the choice of the subcontractor, in order to maximize the adherence of the bid to the contingent needs originated by the geographical return policy.

This industrial need is specific to the space domain in Europe.

IN-05: Quality of the Software Product. The quality of the on-board software (in both functional and non-functional terms) shall be no less than attained with current practices. The novel development approach shall therefore center on a well-defined development process and qualified methodologies and technologies.

This industrial need is common to all other high-integrity domains.

IN-06: Lower Effort Intensiveness of Verification and Validation. In the space domain, no different from other embedded systems industry where correctness of operation is paramount and depends to a large extent on correct hardware-software interaction, Verification and Validation (V&V) activities are by far the largest and most labour-intensive contributor to the software development cost. The new development approach shall therefore strive to contain the labour intensiveness of V&V.

This industrial need is common to all high-integrity domains.

IN-07: Role of Software Suppliers. The market structure of space industry in Europe promoted a diversification of concepts, methodologies and technologies for software the development to the extent that a software supplier can only competitively bid in a single supply chain. ESA wish to allow suppliers to extend their competitiveness, without the need to adapt the software they produce to the specific development policies of each

prime. However, since the amount of ESA satellite projects is limited, the market is not big enough to sustain the presence of several software suppliers. Therefore, a significant change in the market players cannot be realistically expected, but rather a change in the focus of their business activities.

This is a space-specific need that stems from the limitations of the European market.

IN-08: Mitigation of the Impact of Late Requirement Definition or Change. New software requirements or changes to them may occur during development. In the space business the most typical causes of this instability include: late finalization of system design; modifications in the operational strategy; late clarification of system-level contingency or mission management needs. Software modification may also be required to compensate for hardware problems found during system integration. The compression of the software development schedule (as in IN-01) is expected to exacerbate this risk.

Unstable software requirements can disrupt the software development schedule by causing longer time-to-release and occurring when fundamental decisions on software architecture and software design have already been fixed. The novel approach shall be able to mitigate the effects of those hazards.

This industrial need may also hold in other high-integrity domains.

IN-09: Simplification and Harmonization of Fault Detection Isolation and Recovery. Fault Detection, Isolation and Recovery (FDIR) is a system function that handles contingencies that threaten system integrity or its operational objectives. A simplification and hopefully harmonization of the FDIR approach is highly desired as all too often it is tackled in ad-hoc manners that cause massive integration and verification difficulties.

FDIR is problematic in two principal respects: (i) the software element of the FDIR often is attributed to the system team and therefore escapes the visibility, the understanding and the control of the software development team; (ii) FDIR is the software part whose design and finalization occurs later in the development, so that the pressure to integrate it in the software system may cause costly retrofits. These difficulties threaten the attainment of a sound separation of concerns in software development. FDIR is present in all missions and is one of the most important contributors to the complexity of the overall on-board software.

The novel approach should aim at providing a clean separation between the FDIR strategy (that is the policy to be realized to fulfil the applicable system and software requirements) and the mechanisms to realize it. Such a separation is also expected to mitigate the effects of late definition of FDIR requirements (which are part of IN-08).

This industrial need is space specific.

IN-10: Lower Effort for Flight Maintenance. Software flight maintenance is part of the specificity of the space domain. In contrast to similar domains (like civil avionics, or automotive) where it is always possible to physically access the system to perform maintenance operations, after satellite launch, all software maintenance operations have to be performed remotely. Remote software maintenance may be required to adjust configuration parameters, to upload a software patch to mitigate the consequences of faulty hardware, or to correct software bugs. The need to replace in-flight parts of the on-board software is thus inherent to the domain.

Flight maintenance contributes to the operational costs of the satellite (hence to operation budget as opposed to development costs). Reduction of the effort for maintenance operations, as well as a harmonization of the maintenance strategy will decrease the operational cost of maintenance.

In-flight maintenance operations often require a reboot of the on-board software after the upload of a new software image. Reboot constitutes a hazard for the spacecraft, as in that time span it cannot process ground commands. Problems occurring during bootstrap may thus compromise the spacecraft mission or leave the operational team with crippled means to communicate and operate the spacecraft for further troubleshooting. It is therefore highly desirable to minimize the risk of in-flight maintenance operations by updating parts of the software without having to reboot the processor unit.

Another interesting maintenance scenario occurs when managing a constellation of satellites. In general every satellite of a constellation is initially launched with the same on-board software. During operation however, different patches may be applied to distinct satellites, so that their on-board software evolves separately and starts to diverge. Easy recording of the evolution, annotation of the justification for the modification and tracing of the modifications to each version of the on-board software would decrease the maintenance effort through the whole software life cycle.

IN-11: Future Needs. As the novel development approach is targeting future ESA missions, it shall also accommodate support for future software needs. This need is necessarily loose and open ended and follows from the long-ranging goal of the ESA initiative. The most important needs that were identified include: (a) ensuring that the proposed approach shall not be undermined by the shift in hardware technology that may result from, e.g., the advent of multicore processors; (b) allowing the execution of software of different levels of safety in the same processor board.

5 A Software Reference Architecture for Space Applications

It is now in order to summarize the central concepts on which the proposed definition of software reference architecture is based.

The starting point was the derivation of high-level technical requirements from the industrial needs discussed in section 4.2: those were retained as the technical objectives to be achieved by the software reference architecture. Two overarching goals were added to the founding principles of the proposed software reference architecture:

- *composability*, that is achieved when the properties (in the form of assume/guarantee tuples) of individual components are preserved on component composition and deployment on the target system;
- *compositionality*, that is achieved when the properties of the system as a whole can be derived (economically and conclusively) as a function of the properties of its constituting components.

This work seeks to attain those two properties in the form of *composition with guarantees* [24], whereby they can be (1) assured by static analysis, (2) guaranteed throughout implementation, and (3) actively preserved at run time.

The proposed approach was centred on four primary constituents [18]:

1. a *component model* [4], to design the software as a composition of individually verifiable and reusable software units;
2. a *computational model* [23], to relate the design entities of the component model, their execution needs and their non-functional properties for concurrency, time and space, to a framework of analysis techniques which ensures that the architectural description is statically analyzable in the dimensions of interest;
3. a *programming model*, which consists in a tailored subset of a programming language and a set of code archetypes, and is used to ensure that the implementation of the design entities conforms with the semantics, the assumptions and the constraints of the computational model;
4. a *conforming execution platform*, which is in charge of preserving at run time the system and software properties asserted by static analysis and it is able to notify and react to possible violations of them.

In order to increase the industrial applicability of the approach, and as a response to industrial needs of the space domain, the formulation of the approach further included: (a) a development process centred on Model-Driven Engineering (MDE) [21]; (b) the provisions for domain-specific aspects, which complement the approach, yet are consistent to all its underlying principles.

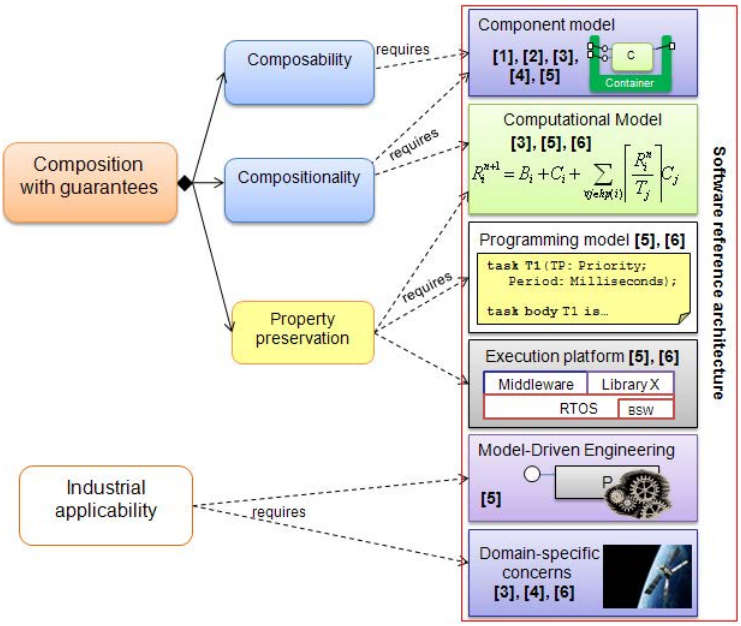

Fig. 2. The goals set for this work and the constituents of the software reference architecture that facilitates their achievement. Square brackets capture the concerns of the software architecture addressed by each individual constituent with the nomenclature shown in figure 1.

Figure 2 recapitulates the goals of our approach and the constituents that needed for their achievements. They collectively form the authors' interpretation of software reference architecture, as originally formulated in [17] and subsequently adopted by the ESA initiative. The same figure depicts which concerns of the software architecture are addressed by each constituent (cf. fig. 1).

Compositionality is earned at the level of the *computational model* and *component model*, as it depends on the adopted body of analysis theories as well as how the architectural entities in use relate to them. Composability is earned at the level of the *component model*. Composability and compositionality, augmented with *property preservation* [24] (i.e., preservation of the analyzed properties at run time), together lead to the achievement of composition with guarantees.

It is thus interesting to observe that, our choice of essential constituents carries an original interpretation of a software reference architecture that achieves properties of vital interest to space stakeholders, and it does so by construction.

5.1 Realization and Validation

As mentioned in section 4.2, ESA funded the COrDeT-2 study project to finalize the definition and prototype realization of the software reference architecture. The component model developed in [19] was adopted as the first constituent of it, together with an implementation built on top of a domain-specific metamodel (short-named "SCM" for "Space Component Model"), equipped with a specialised graphical editor based on the Obeo Designer framework[2].

In a true example of separation of concerns, the designer specifies the on-board software as a set of collaborating components which comprise functional concerns only. It later annotates those components with non-functional requirements, which are then only declaratively specified. Those non-functional requirements are implemented by *containers*, and *connectors*: the former are wrappers taking care of the tasking and synchronization needs of the component; the latter take care of interaction and communication needs (mainly logical and physical distribution of components, data encoding). Containers and connectors are collectively termed "Interaction Layer" in COrDeT-2. Their form depends on the chosen computational model and the associated programming model. In COrDeT-2, in line with preceding studies, the Ravenscar Computational Model [3] was adopted. Thanks to this choice, containers and connectors can be automatically generated following defined property-preserving code patterns [20].

The execution platform of the approach conforms to the description given in section 5. It comprises space-specific services, such as those specified in the "Packet Utilization Standard" [7], which are used for *commandability* and *observability* of the on-board software from ground. The component model offers domain-specific extensions so as to allow the configuration of those space-specific concerns, without breaking the principles or the methodology of the domain-neutral part of the component model.

COrDeT-2 validated the definition and implementation of the reference architecture against an ESA Earth Observation mission reference case[3].

[2] `http://www.obeodesigner.com`
[3] `http://cordet.gmv.com/activity3.htm`

In parallel to COrDeT-2, ESA launched the OSRAc study ("On-board software reference architecture consolidation"[4]) with focus on domain engineering, to gather recurrent solutions and architectural best practices for the functional contents of platform on-board software. OSRAc was to define how to develop reusable software for them using the software reference architecture. The software primes are involved in that study too, to assess the proceedings against a sizeable set of reference missions of interest to ESA and to the French national space agency (CNES).

5.2 Apportionment of Industrial Needs

It is now in order to examine how the software reference architecture as a whole fulfils the industrial needs presented in section 4.2. Table 2 recalls the industrial needs and relates to each of them the architectural constituent, discussed earlier in section 5, that specifically addresses it. Industrial need IN-11 is excluded from the list, as it fell outside of the scope of that phase of the investigation. Another ESA-funded study project (SISTORA[5]) separately addressed those issues; its results will be taken into account in subsequent iterations on the software reference architecture definition.

Table 2. The industrial needs and the constituent of the software reference architecture addressing them. Legend: [CM: Component model; CPM: Computational Model; PM: Programming model; EP: Execution platform; MDE: Model-driven engineering]

ID	Industrial need	Addressed by
IN-01	*Reduced software development schedule*	CM, MDE
IN-02	*Higher cost-effectiveness of software development*	CM, PM, MDE
IN-03	*Support for incremental and parallel software development*	CM, MDE
IN-04	*Multi-team development and product policy*	CM
IN-05	*Quality of the software product*	CM, CPM, PM
IN-06	*Lower effort intensiveness of Verification and Validation*	CPM, PM, MDE
IN-07	*Role of software suppliers*	CM
IN-08	*Mitigation of the impact of late requirement definition or change*	CM, MDE
IN-09	*Simplification and harmonization of FDIR*	CM, EP, MDE
IN-10	*Lower effort for flight maintenance*	CM, PM, EP

The adoption of our component model tailored with domain-specific aspects, and the automation capabilities of MDE (for code generation) serve the purpose of coping with the reduced development schedule for future projects (IN-01). Those same constituents, together with the programming model, are used to increase cost-effectiveness of the development. Our component model supports the specification of non-functional concerns related to tasking, synchronization and communication, separately from the functional concerns and their realization is entirely delegated to code generation [20].

[4] `http://www.congrex.nl/11c22/docs/11C22_ADCSS/10--implementation-stragy--the-sw-perspective.pdf`

[5] `http://congrexprojects.com/docs/12c25_2310/sa1025_jung.pdf?sfvrsn=2`

The proposed component model [19] facilitates incremental development (part of IN-03) and the mitigation of the impact of late requirement changes (IN-08). In doing so, it requires the use of a set of progressively more defined entities: *component types*, for the relationship of individual components to the rest of the software system; *component implementations*, for the concrete realization of components amenable to reuse; *component instances*, to relate to other components for the fulfilment of functional needs and for the declaration of non-functional properties and deployment directives. The ensuing design flow, together with the capability of supporting iterations provided by early model-based analysis, help mitigate the need and impact of late modifications.

As regards parallel development (from IN-03) and decomposition of software to fulfil contingent needs imposed by the the geographical return policy (IN-04), the proposed component model supports parallel development by requiring the definition of well-defined interfaces as part of software decomposition. *Component implementations* are the subcontracting units of the approach. Implementation constraints (in terms of resource consumption bounds) can be set on component implementations prior to outsourcing their realization to suppliers. In that manner the software integrator can better master complex supply chains.

Quality of the software product (IN-05) is achieved by adopting the component model, which informs the design by imposing the design methodology that underpins it; adopting a computational model and a conforming programming model provides for static analyzability of the software product, with the confidence of consistency between analysis and implementation.

The V&V effort is lowered by the use of model-based analysis and code generation. The former permits to converge faster to a system behaviour that fulfills the required non-functional requirements. The latter permits – thanks to the adoption of the programming model – the production of the complete code for tasking, synchronization, communication and interfaces between components. Another important advantage consists in the automated generation of all the test cases needed to confirm the correctness of the generated code, delivering the software engineers from the burden of writing this error-prone part of the software and its associated test suites.

The realization of a component implementation in our component model can be delegated to a software supplier, which can focus on the implementation of functional contents only (as non-functional concerns are dealt only by the software integrator) without needing to adapt the component to the practices of the different software integrator (as required by IN-07); the software integrator can then easily integrate the implemented component in the software system.

A set of mechanisms for the realization of Fault Detection Isolation and Recovery (FDIR) policies are provided by the execution platform. A set of patterns for their use is provided in the domain-specific part of the component model and the code to use them can be automatically generated. The code that implements FDIR concerns is then kept separated from functional code, thus also increasing the reuse potential of components. This contributes to the fulfilment of IN-09.

Finally, the effort for flight maintenance (IN-10) is potentially reduced in our approach by reasoning in terms of components or their constituents instead of memory regions, which earns us better control on the abstraction and granularity of the software

to uplink. Moreover, with support from the execution platform, it will be possible to reload and re-start individual components at run time, without needing to reboot the whole software from an updated software image (which is the current state of practice).

6 Conclusions

This paper reported on the motivation and proceedings of an initiative undertaken by the European Space Agency for the development of a software reference architecture for the on-board software of their future satellite systems. This paper is intended an as enunciation of guiding principles that propose a specific understanding of the concept of software reference architecture and trace its constituents to the industrial needs captured by the relevant domain stakeholders. The reference architecture is being extensively validated in a number of ESA-funded studies with promising results; however only medium-term efforts, much longer than the horizon covered by this work can gather conclusive evidence for or against the proposed approach.

While the reference architecture was a response to needs of the space domain, its stipulation is centred on core domain-neutral concepts and accommodates room for domain-specific extensions or specialization, making it attractive for domains other than space. As an evident confirmation of it, the 4-constituent reference architecture was also successfully adopted by the CHESS project[6], which targeted the industrial needs of telecommunication and railway domains in addition to those of space industry. Each domain covered in CHESS used the same shared component model [19] (extended with domain-specific concerns), and were able to adopt a programming model and execution platform that fit their domain-specific needs. Some of the CHESS results are summarised in [5].

Acknowledgements. The views presented in this paper are the authors' only and do not necessarily engage those of the European Space Agency. This work was supported by the Networking/Partnering Initiative of ESA/ESTEC and by the CHESS project, ARTEMIS JU grant nr. 216682.

References

1. Angelov, S., Grefen, P., Greefhorst, D.: A Classification of Software Reference Architectures: Analyzing Their Success and Effectiveness. In: IEEE/IFIP Conference on Software Architecture and European Conference on Software Architecture, WICSA/ECSA (2009)
2. Barbacci, M., Clements, P., Lattanze, A., Northrop, L., Wood, W.: Using the Architecture Tradeoff Analysis Method (ATAM) to Evaluate the Software Architecture for a Product Line of Avionics Systems: A Case Study. Tech. rep., SEI, Carnegie Mellon University (2003)
3. Burns, A., Dobbing, B., Vardanega, T.: Guide for the Use of the Ada Ravenscar Profile in High Integrity Systems. Technical Report YCS-2003-348, University of York (2003)
4. Chaudron, M., Crnkovic, I.: Component-based software engineering. In: van Vliet, H. (ed.) Software Engineering: Principles and Practice, ch. 18, Wiley (2008)

[6] `http://www.chess-project.org/`

5. Cicchetti, A., Ciccozzi, F., Mazzini, S., Panunzio, M., Puri, S., Vardanega, T., Zovi, A.: CHESS: A Model-Driven Engineering Tool Environment for Aiding the Development of Complex Industrial Systems. In: 27th Int'l Conference on Automated Software Engineering (ASE 2012), pp. 362–365. IEEE/ACM (September 2012) ISBN: 978-1-4503-1204-2
6. Dvorak, D. (ed.): NASA Study on Flight Software Complexity. Tech. rep., Commissioned by the NASA Office of Chief Engineer (2009)
7. European Cooperation for Space Standardization: Space Engineering - Ground systems and operations - Telemetry and telecommand packet utilization, ECSS-E-70-41A (2003)
8. European Cooperation for Space Standardization: Space engineering - Software, ECSS-E-ST-40C (2009)
9. European Cooperation for Space Standardization: Space product assurance - Software product assurance, ECSS-Q-ST-80C (2009)
10. Fortescue, P., Swinerd, G., Stark, J. (eds.): Spacecraft Systems Engineering, 4th edn. Wiley (2011) ISBN: 978-0470750124
11. Gallagher, B.: Using the Architecture Tradeoff Analysis Method to Evaluate a Reference Architecture: A Case Study. Tech. rep., SEI, Carnegie Mellon University (2000)
12. ISO/IEC/(IEEE): Systems and Software engineering - Recomended practice for architectural description of software-intensive systems. ISO/IEC 42010 (IEEE Std) 1471-2000 (2007)
13. Kruchten, P.: The Rational Unified Process: An Introduction, 2nd edn. Addison-Wesley (2000)
14. Larman, C., Basili, V.R.: Iterative and incremental development: A brief history. Computer 36, 47–56 (2003)
15. Lin, C.F.: Modern Navigation, Guidance, And Control Processing. Prentice Hall (1991) ISBN: 978-0135962305
16. Malan, R., Bredemeyer, D.: Defining Non-Functional Requirements. Tech. rep., Bredemeyer Consulting (2001), http://www.bredemeyer.com/pdf_files/NonFunctReq.PDF
17. Panunzio, M.: Definition, realization and evaluation of software reference architecture for use in space application. Ph.D. thesis, University of Bologna, Italy (July 2011), http://www.informatica.unibo.it/ricerca/technical-report/2011/UBLCS-2011-07
18. Panunzio, M., Vardanega, T.: On Component-Based Development and High-Integrity Real-Time Systems. In: Proc. of the 15th International Conference on Embedded and Real-Time Computing Systems and Applications (2009)
19. Panunzio, M., Vardanega, T.: A Component Model for On-board Software Applications. In: Proc. of the 36th Euromicro Conference on Software Engineering and Advanced Applications, pp. 57–64. IEEE (2010)
20. Panunzio, M., Vardanega, T.: Ada Ravenscar Code Archetypes for Component-Based Development. In: Brorsson, M., Pinho, L.M. (eds.) Ada-Europe 2012. LNCS, vol. 7308, pp. 1–17. Springer, Heidelberg (2012)
21. Schmidt, D.C.: Model-Driven Engineering. IEEE Computer 39(2), 25–31 (2006)
22. Software Engineering Institute (editor): Defining Software Architecture - Modern, Classic, and Bibliographic Definitions, SEI - Carnegie Mellon (2010), http://www.sei.cmu.edu/architecture/start/definitions.cfm
23. Vardanega, T.: Development of On-Board Embedded Real-Time Systems: An Engineering Approach. Tech. Rep. ESA STR-260, European Space Agency (1999)
24. Vardanega, T.: Property Preservation and Composition with Guarantees: From ASSERT to CHESS. In: Proc. of the 12th IEEE International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing, pp. 125–132 (2009)